ENTERPRISE WARRANTY HUB

Technical Installation, Architecture, Demonstration Sequence & Operational Guide

DBMS Academic Evaluation • Team 4 • Section AIML A • PES University
Anish Kumar (PES1UG24AM043) • Aarav Yuval B G (PES1UG24AM004)

TABLE OF CONTENTS

1. System Requirements	Page 1	7. Demo Accounts & RBAC Matrix	Page 3
2. Installation Procedure	Page 1	8. Application Workflow Walkthrough	Page 4
3. Environment Configuration	Page 1	9. Step-by-Step DBMS Demonstration	Page 5
4. Docker Container Setup	Page 2	10. Suggested 5–8 Min Evaluation Plan	Page 6
5. Database Migrations & Seeding	Page 2	11. Local Troubleshooting Guide	Page 6
6. Execution Commands (PowerShell)	Page 2	12. Verification Checklist	Page 6

1. System Requirements

The Enterprise Warranty Hub application requires the following software prerequisites for local execution:

- **Operating System:** Windows 10/11, macOS, or Ubuntu Linux (verified on Windows 11 with PowerShell 7 / Windows PowerShell 5.1).
- **Node.js Runtime:** Version 18.18.0 or higher (Node 20 LTS recommended; verified on Node v20.x / v24.x).
- **Package Manager:** npm v9.x or higher (included with Node.js).
- **Container Engine:** Docker Desktop v4.x or Docker Engine with Docker Compose v2 support (WSL2 backend on Windows).
- **Network Ports:** Port `5432` (PostgreSQL) and Port `3000` (Next.js web application).

2. Installation Procedure

Follow these steps to clone and install the project repository:

```
# 1. Clone repository from GitHub
git clone https://github.com/PES1UG24AM004/DBMS-Enterprise-Warranty-Hub.git
cd DBMS-Enterprise-Warranty-Hub

# 2. Install application dependencies
npm install
```

3. Environment Configuration

The repository includes a documented `.env.example` file. For local execution, copy it to `.env.local`:

```
# In PowerShell:
Copy-Item .env.example .env.local
```

The environment variables and their operational purposes are detailed below:

Variable Name	Default Development Value	Description
`DATABASE_URL`	`postgresql://postgres:postgrespassword@localhost:5432/enterprise_warranty`	Full PostgreSQL connection URI used by database clients.
`PGHOST` / `PGPORT`	`localhost` / `5432`	Database server hostname and TCP listening port.
`PGUSER` / `PGPASSWORD`	`postgres` / `postgrespassword`	Standard local PostgreSQL credentials configured in Docker Compose.
`PGDATABASE`	`enterprise_warranty`	Target database catalog name.
`SESSION_SECRET`	`enterprise-warranty-hub-secure-random-academic-key-32chars`	Cryptographic secret used to sign and verify session JWTs.
`NODE_ENV` / `PORT`	`development` / `3000`	Node.js execution environment and HTTP listening port.

> **Security Note:** Never commit `.env.local` to version control. The repository's `.gitignore` explicitly excludes local environment files while preserving `.env.example` for configuration reference.

4. Docker Container Setup

The database infrastructure is fully containerized via `docker-compose.yml` using the official `pgvector/pgvector:pg16` image:

```
services:
  db:
    image: pgvector/pgvector:pg16
    container_name: ewh-postgres
    restart: unless-stopped
    environment:
      POSTGRES_DB: enterprise_warranty
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgrespassword
    ports:
      - "5432:5432"
    volumes:
      - ewh_pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d enterprise_warranty"]
      interval: 3s
      timeout: 3s
      retries: 10

volumes:
  ewh_pgdata:
```

Persistence Guarantee: The named volume `ewh_pgdata` stores database tables, indexes, and write-ahead logs on the host filesystem, ensuring data persists across container restarts, stops, and system reboots.

5. Database Setup (Migrations & Seeding)

Database setup is entirely automated via TypeScript utility scripts located in the `scripts/` directory:

- **`npm run db:migrate`** (**`scripts/migrate.ts`**)**:** Executes `migrations/001_initial_schema.sql`. It enables `uuid-ossp` and `vector` extensions, constructs all 7 tables with check constraints and foreign keys, deploys PL/pgSQL trigger functions (`update_updated_at_column`, `process_warranty_alert`), creates HNSW indexes, and registers dynamic views.
- **`npm run db:seed`** (**`scripts/seed.ts`**)**:** Populates the database with realistic demo accounts (5 users with bcrypt hashes), 18 enterprise assets spanning 8 categories, 18 warranty contracts, 15 maintenance tickets across 6 workflow stages, and 12 historical maintenance records with real 384-dimensional vector embeddings generated locally. Seeding is strictly idempotent.
- **`npm run db:setup`** (**`scripts/setup.ts`**)**:** Executes migration and seeding sequentially in a single command.
- **`npm run model:download`** (**`scripts/init-model.ts`**)**:** Downloads and caches the ONNX MiniLM model locally to prevent runtime delays during evaluation.

6. Running the Application (PowerShell Reference)

All standard administrative commands for Windows PowerShell are listed below:

Action	PowerShell Command	Notes
1. Start Database Container	`npm run db:up`	Runs `docker compose up -d db`.
2. Initialize Database	`npm run db:setup`	Runs migrations and seed data.
3. Cache Embedding Model	`npm run model:download`	Caches 384-dim ONNX model weights.
4. Run Automated Test Suite	`npm run test`	Executes all 39 automated tests.
5. Run Typecheck	`npm run typecheck`	Runs TypeScript compiler checks.
6. Start Development Server	`npm run dev`	Serves app at `http://localhost:3000`.
7. Build Production Bundle	`npm run build`	Compiles optimized production build.
8. Start Production Server	`npm run start`	Runs compiled Next.js production server.
9. Stop Database Container	`npm run db:down`	Stops container (preserves volume).
10. Clean Reset Database	`npm run db:reset`	Drops all tables, re-migrates, re-seeds.

7. Demo Accounts & Role-Based Access Control (RBAC)

The seed dataset provisions five verified demo accounts representing the three supported organizational roles. All accounts share the public academic demo password `Demo123!`:

Account Name	Login Email	System Role	Scope & Intended Demonstration Purpose
Facility Director	`manager@ewh.demo`	`FACILITY_MANAGER`	Full administrative access. Manages all 18 assets, executes custody reassignments, views full audit ledger, and accesses DBMS workbench.
Senior Technician	`technician@ewh.demo`	`TECHNICIAN`	Manages assigned diagnostic queues. Advances maintenance ticket statuses (`Acknowledged` → `Resolved`) and tests vector troubleshooting.
Field Technician	`field.tech@ewh.demo`	`TECHNICIAN`	Secondary technician account used to demonstrate custody handoffs and reassignment transactions between different staff members.
Apex Medical / Vendor	`vendor@ewh.demo`	`VENDOR`	External contractor account. Demonstrates database-level row isolation; restricted strictly to the 4 assets and tickets assigned to this vendor.
Facilities Lead	`aarav@ewh.demo`	`FACILITY_MANAGER`	Secondary manager account for demonstrating multi-manager notification dispatch by the warranty alert trigger.

7.1. One-Click Demonstration Login Shortcuts

To eliminate login typing delays during evaluation, the login card at `http://localhost:3000/login` provides three **1-Click Demo Login** buttons:

- **"Facility Dir." Button:** Automatically populates `manager@ewh.demo` and authenticates as Facility Manager.
- **"Technician" Button:** Automatically populates `technician@ewh.demo` and authenticates as Technician.
- **"Vendor" Button:** Automatically populates `vendor@ewh.demo` and authenticates as Vendor.

These shortcuts execute genuine HTTP `POST /api/auth/login` requests against the PostgreSQL database, validating the password against the bcrypt hash and receiving an HttpOnly session cookie.

7.2. Role-Based Permission Matrix

Capability / Route	Facility Manager	Technician	Vendor
Dashboard Overview (`/dashboard`)	Full Facility Metrics (18 assets)	Technician Work Queue	Vendor Contract Summary (4 assets)
Asset Inventory (`/assets`)	All 18 assets (Add / Edit / Reassign)	Assigned equipment items only	Contracted vendor assets only
Custody Reassignment & Ledger	Full Access (Commit & Rollback)	No Access (403 Forbidden)	No Access (403 Forbidden)
Warranty Management (`/warranties`)	All warranties (Add / Edit / Filter)	Read-only warranty status	Contracted warranty items only
Ticket Progression (`/tickets`)	Open, Assign, Resolve, Close	Acknowledge, Progress, Resolve	Acknowledge & Update Progress
Vector Troubleshooting (`/troubleshoot`)	Full Search Access	Full Search Access	Full Search Access
DBMS Demonstration Page (`/dbms-demo`)	Full Access	No Access (403 Forbidden)	No Access (403 Forbidden)

8. Application Workflow Walkthrough

8.1. Facility Manager Workflow

1. Log in using the **Facility Dir.** 1-click button.
2. Review the **Operational Dashboard**: Observe aggregate counters computed directly from PostgreSQL (18 total assets, 6 expiring within 30 days, 4 expired warranties, 9 active tickets). Notice the category and status distribution bars reflecting real SQL row counts.
3. Navigate to **Assets (** `/assets` **)**: Filter by category (e.g., *Air Conditioners*) or use the search bar. Click on any asset to inspect warranty details, location, and custodial assignee.
4. Demonstrate custody reassignment: Click **"Reassign"** on asset `AST-CL-001`, select a different technician, and commit the transaction. Verify immediate audit logging.
5. Navigate to **Warranties (** `/warranties` **)**: View the dynamically derived warranty states (`ACTIVE`, `EXPIRING_SOON`, `EXPIRED`). Filter by *Expiring Soon* to review the 30-day window items.
6. Navigate to **DBMS Demonstration (** `/dbms-demo` **)**: Walk the evaluator through live transaction ledgers, trigger alert notifications, dynamic view rows, and pgvector extension metadata.

8.2. Technician Workflow

1. Log out and log in using the **Technician** 1-click button (`technician@ewh.demo`).
2. Observe the tailored dashboard: Displays tickets assigned directly to this technician.
3. Navigate to **Maintenance Tickets (** `/tickets` **)**: Click on an assigned ticket (e.g., `TCK-2026-003`). Advance its status from `Assigned` to `In Progress`.
4. Resolve the ticket: Enter technical diagnostic notes (e.g., *"Replaced faulty thermistor and cleared cooling intake manifold"*) and click **"Resolve Ticket"**. The system records the resolution and automatically triggers local embedding generation into `maintenance_records`.
5. Navigate to **Troubleshooting (** `/troubleshoot` **)**: Enter a natural-language query to retrieve historical resolutions for recurring issues.

8.3. Vendor Workflow

1. Log out and log in using the **Vendor** 1-click button (`vendor@ewh.demo`).
2. Observe strict data scoping: The vendor sees only 4 assets and tickets associated with their service contracts.
3. Navigate to **Assets (** `/assets` **)**: Confirm that unassigned enterprise assets (e.g., internal workstations or administrative laptops) are hidden, proving server-side query scoping and authorization.

9. Step-by-Step DBMS Demonstration Sequence

This sequence is specifically designed for demonstrating technical DBMS concepts to an evaluating professor.

Demonstration 1: ACID Database Transactions (Commit & Rollback)

1. Log in as **Facility Manager** and navigate to **Assets (** `/assets` **)**.
2. Locate asset `AST-CL-001` (Dell Precision 5820 Workstation) and click **"Reassign Custody"**.
3. Select **Senior Technician** (`technician@ewh.demo`) as the new assignee and click **"Commit Transaction"**.
4. **Explain the DBMS mechanism:** Point out that PostgreSQL executed `BEGIN`, locked the asset row via `SELECT ... FOR UPDATE`, updated `assets.assigned_to_user_id`, appended an audit record to `asset_maintenance_ledger`, and executed `COMMIT`. Both tables were updated atomically.
5. Now click **"Reassign Custody"** again on the same asset.
6. Check the **"Simulate Failure (Demonstrate ROLLBACK)"** checkbox. Select a different technician and click **"Commit Transaction"**.
7. **Explain the Rollback mechanism:** An error is thrown. The catch block executes `ROLLBACK`. Show that the asset assignee remains unchanged and no ledger row was written, proving atomicity.
8. Open **DBMS Demonstration (** `/dbms-demo` **)**: Show the **Asset Custody Transaction Ledger** table displaying the committed audit trail with timestamps.

Demonstration 2: PL/pgSQL Event Triggers (Warranty Expiration Alerts)

1. On the **DBMS Demonstration** page, scroll to **PostgreSQL Event Trigger: Warranty Alerts**.
2. Show the live notifications generated by trigger `trg_warranty_alert` executing function `process_warranty_alert()`.
3. **Explain the Trigger mechanism:**
   - Whenever a warranty row is inserted or updated, the trigger computes `days_left := NEW.end_date - CURRENT_DATE`.
   - If days_left ≤ 30 and ≥ 0, it dispatches a `WARRANTY_EXPIRING` alert.
   - If days_left < 0, it dispatches a `WARRANTY_EXPIRED` alert.
   - It inserts into `notifications` with `ON CONFLICT (user_id, dedupe_key) DO NOTHING`, guaranteeing deduplication.
4. Click the notification bell in the top navigation bar to show the live unread alert badge and notification tray.

Demonstration 3: Multi-Table Relational SQL Views

1. Navigate to **Warranties (`/warranties`)**: Explain that this interface queries the view `asset_warranty_status`. Show how remaining days are computed dynamically from `CURRENT_DATE`, ensuring values never become stale.
2. On the **DBMS Demonstration** page, scroll to **SQL View: Critical Asset Dashboard**.
3. Show the aggregated rows: Point out how `COUNT(t.id) FILTER` aggregates open tickets while `ARRAY_AGG(t.ticket_code)` collates ticket codes into an array column directly inside PostgreSQL.

Demonstration 4: pgvector Semantic Troubleshooting

1. Navigate to **Troubleshooting (`/troubleshoot`)**.
2. Click on the pre-filled sample prompt: *"laptop overheating and fan spinning at maximum speed"*.
3. Click **"Execute Semantic Vector Search"**.
4. Show the results: The top result is the historical Dell workstation record: *"Laptop CPU severe thermal throttling and fan screaming at full RPM"* with ~63% cosine similarity.
5. Now type a different query: *"water leaking from air conditioner ceiling unit"* and search.
6. Show the result: Correctly ranks the ceiling cassette AC drainage record at the top.
7. **Explain the Vector Architecture:**
   - Model: `onnx-community/all-MiniLM-L6-v2-ONNX` running 100% locally in Node.js.
   - Output: 384-dimensional normalized vector (||v||_2 = 1.0).
   - Index: PostgreSQL HNSW index on `vector_cosine_ops`.
   - Distance calculation: `mr.embedding <=> $1::vector` (cosine distance).
   - Deduplication: Analytical SQL window function prevents duplicate cards.

10. Suggested 5–8 Minute Presentation Sequence

Timestamp	Topic	Key Talking Points & Action
0:00 – 1:00	Project Intro & RBAC Login	Introduce EWH (Team 4, AIML A, PES University). Click "Facility Dir." 1-click login. Explain raw SQL architecture (node-postgres, no ORMs).
1:00 – 2:00	Dashboard & SQL Views	Present live metrics (18 assets, 6 expiring, 4 expired, 9 tickets). Explain view `asset_warranty_status` computing real-time status dynamically.
2:00 – 3:30	ACID Transactions (Assets)	Go to `/assets`. Reassign asset `AST-CL-001` to Technician & commit. Then check "Simulate Failure" and show atomic `ROLLBACK`. Show ledger audit trail.
3:30 – 4:30	PL/pgSQL Triggers	Navigate to `/dbms-demo`. Explain trigger function `process_warranty_alert()`, 30-day window evaluation, and deduplication via unique constraint.
4:30 – 6:00	pgvector Semantic Search	Go to `/troubleshoot`. Run search for "laptop overheating". Explain local MiniLM ONNX (384-d), HNSW index, cosine distance (`<=>`), and deduplication window function.
6:00 – 7:00	Role Scoping (Vendor)	Sign out, log in as Vendor. Show that Vendor sees only 4 assigned assets vs Manager's 18, proving database-level data isolation.
7:00 – 8:00	Test Suite & Conclusion	Highlight 39/39 passing automated tests, container volume persistence, and invite questions from the evaluator.

11. Local Troubleshooting Guide

Observed Symptom	Probable Cause	Resolution Procedure
`dial tcp 127.0.0.1:5432: connect: connection refused`	Docker Desktop is not running, or PostgreSQL container `ewh-postgres` is stopped.	Launch Docker Desktop, wait for engine to initialize, and run: `npm run db:up`. Confirm with `docker compose ps`.
`Port 5432 is already allocated`	A local native PostgreSQL instance is running on the host machine.	Stop local Windows service: `Stop-Service postgresql*` in PowerShell, or stop the competing container.
`Port 3000 is already in use`	Another Next.js or Node.js process is occupying port 3000.	Run `Get-Process node | Stop-Process -Force` or specify a custom port: `npm run dev -- -p 3001`.
`relation "users" does not exist`	Database container was created fresh but migrations were not executed.	Run `npm run db:setup` to execute migrations and seed data.
Model download timeout / offline execution	First semantic search attempts to download ONNX model without active internet connection.	Run `npm run model:download` once while connected to download model weights (~90MB) to local cache.
Corrupted / Inconsistent Database State	Accidental partial deletions during testing.	Run `npm run db:reset` to cleanly drop all tables, re-run migrations, and re-seed the full catalog.

12. Final Release Verification Checklist

Before university evaluation, verify that all release gates are satisfied:

- [x] **Docker PostgreSQL Container:** Running and healthy on port 5432 with pgvector extension enabled.
- [x] **TypeScript Compilation:** `npm run typecheck` passes with code 0 (zero errors).
- [x] **Automated Test Suite:** `npm run test` reports 39 / 39 tests passed (0 failed).
- [x] **Production Build:** `npm run build` successfully compiles all 27 App Router routes.
- [x] **Database Seeding:** 5 demo accounts, 18 assets, 18 warranties, 15 tickets, 12 vector records seeded idempotently.
- [x] **ACID Transactions:** Reassignment commit and rollback verified in live UI and automated tests.
- [x] **Triggers & Views:** `process_warranty_alert()` and views `asset_warranty_status` & `critical_asset_dashboard` verified.
- [x] **Local pgvector Search:** 384-dimensional cosine distance queries functioning with deduplication.
- [x] **Documentation:** `docs/Project_Report.pdf` and `docs/Technical_and_Demo_Guide.pdf` compiled in grayscale academic style with zero headers or footers.